

C# / .NET Learning Roadmap

Topic Notes — Section 0: Foundations Before Code

Topics: 0.1 through 0.8

Date: 29 June 2026

Section 0 — Foundations Before Code

This section covers the foundational knowledge required before writing any C# code. EF Core and ASP.NET Core both assume prior knowledge of HTTP, REST, and basic SQL/relational database concepts that are easy to take for granted but essential for a true zero-knowledge starting point.

0.1. What is HTTP — Request/Response Cycle, Statelessness

Builds on: Nothing prior — this is your true starting point. Everything in ASP.NET Core and EF Core sits on top of this.

What is HTTP and why does it exist?

HTTP (HyperText Transfer Protocol) is the agreed-upon language that browsers, mobile apps, and servers use to talk to each other over the internet. Without a shared protocol, a browser built by Google couldn't talk to a server built by Microsoft. HTTP is that contract.

Think of it like ordering at a restaurant: you (client) hand a written order slip (request) to the waiter, who takes it to the kitchen (server). The kitchen prepares a response and sends it back. Both sides agree on the format of the slip and the response — that agreement is HTTP.

The Request/Response Cycle

Every HTTP interaction follows this exact pattern — no exceptions:

- Client sends a Request to a server URL
- Server processes it and sends back a Response
- The connection's job is done

A request contains:

- **Method** — what action to perform (GET, POST, etc. — covered in 0.2)
- **URL** — which resource to act on (e.g. /api/users/42)
- **Headers** — metadata (e.g. Content-Type: application/json, auth tokens)
- **Body** — optional data payload (e.g. JSON you're sending to create a user)

A response contains:

- **Status code** — result of the operation (covered in 0.3)
- **Headers** — metadata about the response
- **Body** — the returned data (e.g. a JSON user object)

Minimal example — what actually travels over the wire:

```
-- REQUEST --
GET /api/users/42 HTTP/1.1
Host: myapp.com
Authorization: Bearer eyJ...

-- RESPONSE --
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{ "id": 42, "name": "Sathish" }
```

No magic — just formatted text sent back and forth.

Statelessness — the critical part

HTTP is stateless: the server remembers nothing about you between requests. Every request must be self-contained and carry all the information the server needs.

This surprises people coming from desktop app thinking. If you log in on request #1, the server has already forgotten you by request #2 — unless your request #2 carries proof of login (e.g. a token in the header).

Why stateless? Scalability. If the server stored session memory, you'd have to hit the same server every time. Statelessness lets any server in a cluster handle any request.

In .NET specifically: This is why JWT tokens exist (section 15.3) — they're how you carry identity on every request without the server storing session state.

Where this shows up in real .NET codebases

- Every ASP.NET Core controller action is one half of a request/response cycle. When you write [HttpGet] on a method, you're defining what happens when a GET request hits that route.
- HttpClient (section 16.1) — when your .NET backend calls another API, it's making HTTP requests programmatically.
- Middleware pipeline (section 13.6) — ASP.NET Core processes every incoming HTTP request through a chain of middleware before it reaches your controller.
- Authentication (section 15) — entirely built around the stateless constraint. Tokens exist because HTTP doesn't remember you.

Common Misconceptions

- **The server keeps track of what I'm doing.** It doesn't. Each request is a stranger walking in cold. State is the client's responsibility to carry (via cookies, tokens, query params).
- **HTTP is only for browsers.** No — your .NET backend calling a payment API, a mobile app hitting your API, a CLI tool hitting GitHub — all HTTP.
- **HTTPS is different from HTTP.** HTTPS is HTTP with encryption (TLS) layered on top. The request/response structure is identical; it's just scrambled in transit.
- **The body is always JSON.** The body can be JSON, XML, form data, a file, or nothing. The Content-Type header tells the receiver what format to expect.

Self-check — you should now be able to explain:

- What the two sides of an HTTP interaction are and what each sends
- The four components of a request and three of a response
- Why HTTP is called stateless and what real consequence that has

- Why a JWT token needs to be sent on every request, not just the login one

0.2. HTTP Verbs (GET, POST, PUT, PATCH, DELETE)

Builds on: 0.1 — These verbs are the "method" field of every HTTP request you learned about. They tell the server what action to perform on a resource.

What are HTTP Verbs and why do they exist?

A URL identifies what resource you're talking about (/api/users/42). The verb tells the server what to do with it. Without verbs, every action would need a different URL — /getUser, /deleteUser, /updateUser — chaos with no standard.

Verbs bring convention: one URL (/api/users/42), multiple actions depending on the verb.

The 5 Verbs You Need

Verb	Meaning	Has Body?	Real example
GET	Fetch a resource	No	Get user #42's profile
POST	Create a new resource	Yes	Submit a new user registration
PUT	Replace a resource entirely	Yes	Overwrite user #42's full record
PATCH	Update part of a resource	Yes	Change only user #42's email
DELETE	Remove a resource	No	Delete user #42

GET

Retrieve data. Must not change anything on the server — it's read-only by convention. Safe to call multiple times with identical results.

```
GET /api/users/42
```

Returns user 42's data. Nothing is modified.

POST

Create something new. The server decides the new resource's ID and returns it.

```
POST /api/users
Body: { "name": "Sathish", "email": "s@zoho.com" }
```

Server creates the user, assigns ID 43, returns the created object (or its location).

PUT vs PATCH — the one people always confuse

PUT = send the entire updated object. Whatever you omit gets wiped.

```
PUT /api/users/42
Body: { "name": "Sathish Kumar", "email": "sk@zoho.com", "role": "admin" }
```

All three fields must be present. If you omit role, it gets nulled out on the server.

PATCH = send only what changed. The rest stays untouched.

```
PATCH /api/users/42
Body: { "email": "new@zoho.com" }
```

Only email updates. Name and role are unaffected.

DELETE

Remove the resource. Typically no body, no response body either — just a 204 No Content status.

```
DELETE /api/users/42
```

Realistic mini-scenario — ASP.NET Core controller

This is exactly how these verbs map to controller actions in .NET:

```
[ApiController]
[Route("api/users")]
public class UsersController : ControllerBase
{
    [HttpGet("{id}")]           // GET /api/users/42
    public IActionResult Get(int id) { ... }

    [HttpPost]                  // POST /api/users
    public IActionResult Create(CreateUserDto dto) { ... }

    [HttpPut("{id}")]          // PUT /api/users/42
    public IActionResult Replace(int id, UpdateUserDto dto) { ... }

    [HttpPatch("{id}")]        // PATCH /api/users/42
    public IActionResult PartialUpdate(int id, PatchUserDto dto) { ... }

    [HttpDelete("{id}")]       // DELETE /api/users/42
    public IActionResult Delete(int id) { ... }
}
```

One resource (/api/users/{id}), five verbs, five distinct operations — clean and predictable.

Common Misconceptions

- **POST is for everything — login, submit, search.** Many beginners use POST for all non-GET actions. Technically it works, but it breaks REST conventions that your teammates and API consumers expect.
- **PUT and PATCH are interchangeable.** They're not. Sending a PATCH payload to a PUT endpoint can silently null out fields the user didn't intend to clear.
- **GET can have a body.** Technically allowed by the spec, but effectively forbidden in practice. Many proxies and servers strip GET bodies. Use query parameters instead: GET /api/users?role=admin.
- **DELETE always returns 200.** Convention is 204 No Content — success, but nothing to return. Some APIs return 200 with a confirmation message, but 204 is cleaner.
- **Verbs are enforced by HTTP itself.** They're not. Nothing stops a server from deleting data on a GET request. The verbs are a convention — REST discipline is on you as the developer to honor.

Practice Problem

Design the verb + URL for these 4 operations on a products resource:

- Get all products
- Create a new product
- Update only the price of product #7
- Delete product #7

Write them as VERB /path — no code needed.

Self-check — you should now be able to explain:

- What each of the 5 verbs means and when to use each
- The specific difference between PUT and PATCH with an example
- Why GET should never modify server state
- How verbs map to [HttpGet], [HttpPost] etc. in an ASP.NET Core controller

0.3. HTTP Status Codes (2xx, 3xx, 4xx, 5xx)

Builds on: 0.1 (response structure), 0.2 (verbs trigger different codes — DELETE returns 204, POST returns 201).

What and why?

The status code is the server's one-line verdict on what happened. Without it, the client would have to parse the response body every time just to know if something worked. The code gives an immediate, unambiguous signal.

The Four Families

Range	Meaning	Who's responsible
-----	-----	-----
2xx	Success	—
3xx	Redirection	Client must go elsewhere
4xx	Client error	Caller's fault
5xx	Server error	Server's fault

2xx — Success

Code	Name	When to use
-----	-----	-----
200	OK	GET, PUT, PATCH succeeded — body contains result
201	Created	POST succeeded — new resource was created
204	No Content	DELETE succeeded — nothing to return

// In ASP.NET Core controllers:

```
return Ok(user);           // 200 with body
return Created($"api/users/{user.Id}", user); // 201 with location
return NoContent();        // 204, no body
```

3xx — Redirection (awareness only at this stage)

Code	Name	When
----- ----- -----		
301	Moved Permanently	Resource URL has changed forever
302	Found (temp redirect)	Temporary redirect

You'll rarely write 3xx responses in a REST API. More relevant for web pages and old MVC apps.

4xx — Client Errors (most important for API dev)

Code	Name	When
----- ----- -----		
400	Bad Request	Malformed input, validation failure
401	Unauthorized	Not authenticated (no/invalid token)
403	Forbidden	Authenticated but not allowed
404	Not Found	Resource doesn't exist
409	Conflict	State conflict (e.g. duplicate email on register)
422	Unprocessable Entity	Syntactically valid but semantically wrong

5xx — Server Errors

Code	Name	When
----- ----- -----		
500	Internal Server Error	Unhandled exception on server
503	Service Unavailable	Server overloaded or down

A 500 in production means your server crashed — something you need to fix. A 4xx means the client did something wrong.

Realistic scenario

```
[HttpGet("{id}")]
public IActionResult GetUser(int id)
{
    var user = _repo.FindById(id);

    if (user == null)
        return NotFound(); // 404

    return Ok(user); // 200
}

[HttpPost]
public IActionResult CreateUser(CreateUserDto dto)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState); // 400

    var user = _service.Create(dto);
    return Created($"/api/users/{user.Id}", user); // 201
}
```

Common Misconceptions

- **401 and 403 are the same.** 401 = "I don't know who you are." 403 = "I know who you are, but you can't do this." A user without a token gets 401. A regular user trying to access an admin endpoint gets 403.
- **Just return 200 for everything and put the error in the body.** Some legacy APIs do this — it's an antipattern. Clients (and .NET's HttpClient) check status codes to decide how to handle responses.
- **500 is acceptable in production.** A 500 means an unhandled exception escaped. Every 500 should be logged, investigated, and eliminated.
- **404 means the route doesn't exist.** It can also mean the route exists but the specific resource wasn't found (e.g. /api/users/999 where user 999 doesn't exist). Both are 404.

Self-check:

- Difference between 401 and 403
- Which 2xx code belongs to POST vs DELETE vs GET
- What a 500 tells you vs a 400

0.4. What is REST — Resources, Endpoints, Conventions

Builds on: 0.1, 0.2, 0.3 — REST is the design philosophy that ties all three together into a coherent API structure.

What is REST and why does it exist?

REST (Representational State Transfer) is a set of conventions for designing HTTP APIs that are predictable, consistent, and easy to consume. It's not a protocol or a library — it's a style guide.

Without REST conventions, every API team invents their own URL structure. REST gives a shared vocabulary so any developer can look at your API and instantly understand it.

The Core Idea: Resources

Everything in REST is a resource — a noun, not a verb. You identify resources with URLs and act on them with HTTP verbs.

```
# Non-REST (verb in URL – bad)
GET  /getUser?id=42
POST /deleteUser?id=42
POST /createProduct

# REST (noun in URL – good)
GET    /api/users/42
DELETE /api/users/42
POST   /api/products
```

REST Conventions

Use nouns, plural:

```
/api/users           (collection)
```


`/api/users/42` (specific item)
`/api/users/42/orders` (related sub-resource)

Combine with verbs for full meaning:

GET	<code>/api/users</code>	-> list all users
POST	<code>/api/users</code>	-> create a user
GET	<code>/api/users/42</code>	-> get user 42
PUT	<code>/api/users/42</code>	-> replace user 42
PATCH	<code>/api/users/42</code>	-> partial update user 42
DELETE	<code>/api/users/42</code>	-> delete user 42

Use query parameters for filtering, not new routes:

GET `/api/users?role=admin&active=true`

What REST is NOT

REST doesn't require:

- JSON (it's just the most common format)
- Any specific framework
- Authentication of any particular kind

REST is stateless by design — directly inheriting HTTP's statelessness from 0.1.

Where it shows up in .NET

Every ASP.NET Core Web API you build follows REST conventions. When you define routes like `[Route("api/users")]` and use `[HttpGet]`, `[HttpPost]` etc., you're implementing REST.

Common Misconceptions

- **REST and HTTP are the same thing.** HTTP is the transport protocol. REST is a design style that uses HTTP. You can use HTTP without REST.
- **Any HTTP API is a REST API.** Only if it follows the conventions. An API that uses POST for everything and puts verbs in URLs is just an HTTP API, not a REST API.
- **REST requires a specific response format.** No — REST says nothing about JSON or XML. JSON became dominant by convention, not rule.

Self-check:

- Design a RESTful URL structure for a books resource with CRUD operations
- Explain why `/api/getBooks` violates REST but `/api/books` doesn't

0.5. What is a Relational Database — Tables, Rows, Columns

Builds on: Nothing prior — this is a parallel foundation track alongside HTTP, needed for EF Core (section 14).

What and why?

A relational database stores data in a structured, organized way so it can be queried efficiently and relationships between data can be maintained. Without a database, your data lives only in memory — gone when the server restarts.

"Relational" means data is organized into tables that can relate to each other.

The Three Core Concepts

Table = a category of data (like a spreadsheet tab)

Row = one record/instance in that category

Column = one attribute/property of that record

Table: Users

+-----+-----+-----+-----+			
Id	Name	Email	
+-----+-----+-----+-----+			
1	Sathish	s@zoho.com	
2	Priya	p@zoho.com	
3	Karthik	k@zoho.com	
+-----+-----+-----+-----+			

- The table is Users
- Each row is one user
- Id, Name, Email are columns

Why "relational"?

Tables can reference each other. An Orders table can reference the Users table — "this order belongs to user #1":

Table: Orders

+-----+-----+-----+-----+			
Id	Product	UserId	<- references Users.Id
+-----+-----+-----+-----+			
1	Keyboard	1	<- Sathish's order
2	Monitor	1	<- also Sathish's
3	Mouse	2	<- Priya's order
+-----+-----+-----+-----+			

This relationship is what makes it relational — and why you don't need to duplicate Sathish's name/email in every order row.

In .NET

EF Core maps your C# classes directly to tables. A User class becomes the Users table, its properties become columns, its instances become rows. This is the conceptual bridge you'll use constantly in section 14.

Common Misconceptions

- **A database is just a big spreadsheet.** A spreadsheet has no enforcement of rules, relationships, or data types. A relational database enforces all three.

- **Each table should store everything.** That's the opposite of good design. Split related but distinct data into separate tables and link them — that's the whole point of relational.

Self-check:

- Given a blogging app, name at least 3 tables you'd need and what columns each would have
- Explain what "relational" means in one sentence

0.6. Primary Keys and Foreign Keys

Builds on: 0.5 — keys are how relational databases uniquely identify rows and enforce relationships between tables.

Primary Key

Every table needs a way to uniquely identify each row. That's the primary key — a column (or combination of columns) guaranteed to be unique and never null.

```
Table: Users
+----+-----+
| Id | Name   |  <- Id is the primary key
+----+-----+
| 1  | Sathish |
| 2  | Priya   |
+----+-----+
```

Id is almost universally used. Databases auto-increment it — you insert a row and the DB assigns the next available integer.

Foreign Key

A foreign key is a column in one table that references the primary key of another table. It's how relationships are enforced at the database level.

```
Table: Orders
+----+-----+-----+
| Id | Product | UserId |  <- UserId is a foreign key -> Users.Id
+----+-----+-----+
```

The database enforces that UserId must match an existing Id in Users. You can't insert an order for a user that doesn't exist — the DB rejects it. This is called referential integrity.

In EF Core (preview)

```
public class User
{
    public int Id { get; set; }           // primary key (EF detects by convention)
    public string Name { get; set; }
    public List<Order> Orders { get; set; } // navigation property
}

public class Order
```

```
{
    public int Id { get; set; }
    public int UserId { get; set; }    // foreign key
    public User User { get; set; }    // navigation property
}
```

EF Core reads this structure and creates the right constraints in the database automatically.

Common Misconceptions

- **I can use Name or Email as a primary key.** You can, but you shouldn't. Names aren't unique, emails can change. Use a surrogate integer or GUID Id that has no business meaning and never changes.
- **Foreign keys are optional.** Without them, you can insert orphaned records (an Order pointing to a non-existent User). Foreign keys enforce consistency — always use them.
- **Primary key = auto-increment integer, always.** Common but not required. GUIDs are used when you need globally unique IDs (distributed systems, public-facing IDs you don't want to be guessable).

Self-check:

- What happens at the DB level if you try to delete a User who has existing Orders (with a foreign key constraint)?
- Why is Email a poor primary key choice?

0.7. Basic SQL — SELECT, INSERT, UPDATE, DELETE, WHERE, JOIN basics

Builds on: 0.5, 0.6 — SQL is the language you use to interact with the tables and relationships you just learned about.

What is SQL?

SQL (Structured Query Language) is the standard language for talking to relational databases. Even when using EF Core, knowing SQL matters — EF Core generates SQL, and when something goes wrong or performs poorly, you need to read what it generated.

SELECT — Read data

```
-- All columns, all rows
SELECT * FROM Users;

-- Specific columns
SELECT Id, Name FROM Users;

-- With condition
SELECT * FROM Users WHERE Id = 1;

-- Multiple conditions
```

```
SELECT * FROM Users WHERE Name = 'Sathish' AND Email LIKE '%zoho.com';
```

INSERT — Create a row

```
INSERT INTO Users (Name, Email)
VALUES ('Sathish', 's@zoho.com');
-- Id is auto-assigned by the DB
```

UPDATE — Modify existing rows

```
UPDATE Users
SET Email = 'new@zoho.com'
WHERE Id = 1;
```

Warning: Always include WHERE — without it, every row gets updated.

DELETE — Remove rows

```
DELETE FROM Users
WHERE Id = 1;
```

Warning: DELETE FROM Users with no WHERE deletes everything.

WHERE — Filtering

```
SELECT * FROM Orders WHERE UserId = 1;
SELECT * FROM Users WHERE Name LIKE 'S%'; -- starts with S
SELECT * FROM Orders WHERE Amount > 1000;
```

JOIN — Combine related tables

This is where "relational" pays off. JOIN lets you query across tables using the foreign key relationship:

```
-- Get all orders with the user's name attached
SELECT Users.Name, Orders.Product
FROM Orders
JOIN Users ON Orders.UserId = Users.Id;
```

Result:

```
+-----+-----+
| Name   | Product |
+-----+-----+
| Sathish | Keyboard |
| Sathish | Monitor  |
| Priya   | Mouse    |
+-----+-----+
```

In .NET

EF Core translates LINQ queries into SQL. When you write:

```
var users = context.Users.Where(u => u.Name == "Sathish").ToList();
```

EF Core generates:

```
SELECT * FROM Users WHERE Name = 'Sathish'
```

Knowing SQL helps you verify EF Core is generating efficient queries and debug when it isn't.

Common Misconceptions

- **EF Core means I never need to know SQL.** EF Core hides SQL but doesn't eliminate it. N+1 problems, slow queries, and migration issues all require SQL knowledge to diagnose.
- **UPDATE without WHERE is fine if I want to update everything.** Almost never the right answer in production — always double-check your WHERE clause.
- **JOIN is complicated.** The concept is simple: match rows from two tables where a column value is equal. Start with basic INNER JOINS and build up.

Practice Problem

Write SQL to get all orders placed by the user named 'Priya', showing the product name and user email. Use the Users and Orders tables from earlier examples (assume Orders also has an Amount column).

Self-check:

- Write a SELECT that gets all users whose email ends in @zoho.com
- Explain what a JOIN does in one sentence
- What's the danger of UPDATE or DELETE without WHERE?

0.8. What is an API — Client/Server Model, JSON as Data Format

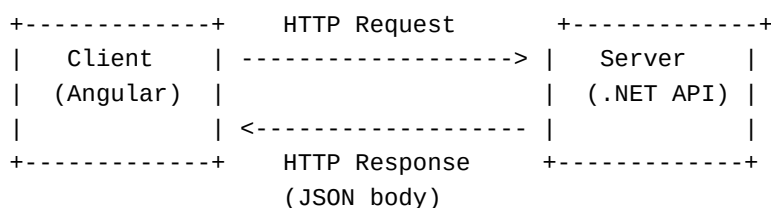
Builds on: 0.1–0.4 (HTTP + REST) and implicitly 0.5–0.7 (the server usually has a DB behind it). This topic ties the full picture together.

What is an API?

API stands for Application Programming Interface. In the web context, it's a server that exposes data and operations over HTTP so other applications can use them — without knowing how the server works internally.

Your Angular frontend doesn't know or care whether your .NET backend uses SQL Server or PostgreSQL, or whether it's written in C# or Go. It just makes HTTP requests and gets JSON back. That contract is the API.

The Client/Server Model



Client = anything that makes requests (browser, mobile app, another server, CLI tool)

Server = listens for requests, processes them, returns responses

They are completely separate — different codebases, different deployments, communicating only through HTTP.

JSON — The Universal Data Format

JSON (JavaScript Object Notation) is the de facto standard for API data exchange. It's human-readable, language-agnostic, and maps naturally to objects.

```
{
  "id": 42,
  "name": "Sathish",
  "email": "s@zoho.com",
  "isActive": true,
  "orders": [
    { "id": 1, "product": "Keyboard" },
    { "id": 2, "product": "Monitor" }
  ]
}
```

In .NET, `System.Text.Json` serializes C# objects to JSON and deserializes JSON back to C# objects automatically:

```
// C# object -> JSON (serialization)
var json = JsonSerializer.Serialize(user);

// JSON -> C# object (deserialization)
var user = JsonSerializer.Deserialize<User>(json);
```

ASP.NET Core does this automatically when your controller returns an object — you don't call `Serialize` manually in most cases.

The Full Picture

Putting all of Section 0 together — a real request flow:

1. Angular makes: `GET /api/users/42` (HTTP verb + URL from 0.2, 0.4)
2. .NET API receives the request
3. Controller action runs, queries the DB (SQL from 0.7)
4. Returns a C# `User` object -> auto-serialized to JSON
5. Response arrives with status 200 (0.3) and JSON body
6. Angular reads the JSON and displays the data

Every section in 0.1–0.7 has a role in this single flow.

Common Misconceptions

- **API means web API.** APIs exist between any two software components — a library's public methods are an API. "API" in job postings usually means HTTP/REST API, but the term is broader.
- **JSON is a JavaScript thing.** The name is misleading. JSON is language-agnostic — .NET, Python, Java, Go all use it natively.
- **The frontend and backend must be on the same server.** No — they're typically separate deployments entirely. The client/server model only requires they agree on the HTTP contract.

- **API = database.** The API is the layer in front of the database. The API decides what the client is allowed to see and do; the database just stores the data.

Self-check:

- Trace a complete Angular -> .NET API -> Database -> response flow in your own words
- What does serialization mean, and where does it happen in an ASP.NET Core response?
- Why is JSON language-agnostic even though it has "JavaScript" in the name?

Section 0 complete. You now have the full conceptual foundation that every topic in Sections 1–17 builds on.